

Finite Network Improvements for the Exact Discrete Fourier Transform

Ryan Shea

October 9, 2026

We give a reusable conditional transfer from finite complex tensor networks to the exact discrete Fourier transform, together with versioned network witnesses. The current recorded all-length bound is

$$T(n) = O\left(n(\log n)^{1-\delta}\right), \quad \delta = 0.000754736.$$

The selected finite witness has strict tensor saving $a = 0.0007547361$. The selected bound is the upstream theorem from Jacob Sussman’s five-stage framework and Chafik Boukhalfa’s stronger instantiation of the community circuit, reproduced from pinned project sources with the standard Lean axioms and theorem/definition comparisons. Independent human mathematical review remains pending. The reusable written transfer argument in this paper is distinct from that imported formal proof. This paper separates the reusable transfer theorem, the selected construction, and the history of incorporated improvements. No practical FFT speedup is claimed.

1. Scope and the finite-network interface

This paper addresses OpenAI Problem #130. Its reusable argument transfers finite tensor-network improvements to an all-length exact discrete Fourier transform. Each selected network is a separate, pinned result record. A passing numerical certificate alone is not a transfer proof. The result history records which networks have a written transfer and which remain candidates.

We use the exact-complex-arithmetic model of OpenAI’s *An explicit power saving for the exact discrete Fourier transform* [O]. Complex field operations have unit cost, coefficients and intermediate magnitudes are unrestricted, and a specified root of unity of order less than $1024n^3$ is supplied. Integer operations and random access on a fixed number of $O(\log(n+2))$ -bit words have unit cost. Scalar preparation, index construction, initialization and data movement are charged. This is not a bit-complexity, numerical-stability, bounded-coefficient or practical FFT claim.

The fixed kernel is

$$C = \frac{1}{2} \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix}, \quad C^2 = X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Finite-network contract. Fix integers $m \geq 2$, $W \geq 1$, and nonnegative integer multiplicities n_r for $1 \leq r < m$. For every column count $f \geq 1$, require a constructively specified finite algorithm

that applies $C^{\otimes mf}$ to **all W arbitrary complex input arrays**, including arrays used as dirty auxiliary registers. It must have the following properties.

1. Its positive-width operations consist of n_r calls on rf binary axes in one array, independently on every fiber of the remaining axes. Each such call is $C^{\otimes rf}$, up to explicitly implemented adapters.
2. Everything outside these calls costs at most $AW2^k$ for $k = mf + b$, $0 \leq b < m$, with arbitrary spectator axes allowed. The constant A is independent of k and f . Temporary copies, scalar gates, phase corrections, layout passes and their inverses are included.
3. The schedule and fixed tables have constructive finite preparation procedures. Serial execution uses $O(W2^k)$ live complex registers and addresses represented by a fixed number of $O(k+1)$ -bit words.
4. The specified input-output map is exact on arbitrary data and scratch. The histogram counts actual recursive calls, including copied-register transformations and endpoint corrections. A rank budget or aggregate scalar identity by itself does not satisfy this condition.

This contract makes no particular choice of number of stages, producer DAG, frame system or network optimizer. Different witnesses may discharge it by different arguments. A fixed finite cover is permitted: if a cover has V vertices and counts $W_0, n_r^{(0)}$ per vertex, the actual values are $W = VW_0$, $n_r = Vn_r^{(0)}$. The factor cancels in the normalized moment, but the algorithm still allocates and routes the full finite stock. Rational per-vertex counts require an explicitly justified common integer replication; they are not fractional physical registers.

2. A reusable transfer theorem

Conditional transfer theorem. Suppose the finite-network contract holds and, for a rational $0 < a < 1$,

$$\Psi(1-a) = \frac{1}{W} \sum_{r=1}^{m-1} n_r \left(\frac{r}{m}\right)^{1-a} < 1. \quad (2.1)$$

Then the network supplies a uniform arbitrary-input tensor algorithm of cost

$$O\left(2^k(k+1)^{1-a}\right). \quad (2.2)$$

Under [O]’s Fourier compiler and all-length reduction, every fixed $0 < \delta < a$ gives

$$T(n) = O\left(n(\log n)^{1-\delta}\right) \quad (2.3)$$

in the model of Section 1. The constants and eventual thresholds may depend on the entire finite witness. This theorem is conditional on the actual algorithmic contract, not solely on the finite inequality (2.1).

2.1 Incomplete batches and unequal child widths

Let $T(k)$ denote the simultaneous cost on W arbitrary arrays, and set $t(k) = T(k)/(W2^k)$. Handle the fixed range $k < m$ directly. For $k \geq m$, let $f = \lfloor k/m \rfloor$ and process the fewer than m leftover

axes directly. A width- r call has 2^{k-rf} fibers. Pack complete groups of W fibers and recurse; apply the ordinary coordinate algorithm to the fewer than W leftover fibers. Thus

$$T(k) \leq AW2^k + \sum_r n_r \left\lfloor \frac{2^{k-rf}}{W} \right\rfloor T(rf) + B(W-1) \sum_r n_r rf 2^{rf}. \quad (2.4)$$

Every child has $r < m$, so

$$rf 2^{rf-k} \leq rf 2^{-(m-r)f} \leq r.$$

The last line of (2.4), after normalization, is bounded by a constant independent of k . Consequently

$$t(k) \leq A' + \frac{1}{W} \sum_r n_r t(r \lfloor k/m \rfloor). \quad (2.5)$$

Put $\theta = 1 - a$. Choose K for the finite base range and so that $K(1 - \Psi(\theta))m^\theta \geq A'$. Since $rf \leq (r/m)k < k$, strong induction gives

$$t(k) \leq A' + Kk^\theta \Psi(\theta) \leq Kk^\theta.$$

A single requested transform initializes $W - 1$ additional arrays and keeps its own output. Because W is a fixed integer, this remains $O(2^k(k+1)^\theta)$. Serial children have at most half their parent's array length, so reusable buffers give a geometrically summable $O(W2^k)$ live stock. The axis count contracts by the fixed factor $\max r/m < 1$, giving $O(\log(k+2))$ stack depth.

2.2 Propagation to every Fourier length

The proof of [O], Proposition 4.2 consumes the arbitrary-input tensor bound, including scalar and address work. Substitution of (2.2) gives its working-length bound

$$O\left(R(\ell+1)^\theta(1+\log r_{\max})^4\right) + \text{poly}(\ell, r_{\max}, \log(R+2)).$$

Its exact-width Fourier compiler, denominator conditions and supplied root are unchanged. Its small-prime working lengths, CRT permutations and charged chirp convolution, with $\ell = \Theta(\log n / \log \log n)$, yield

$$T(n) = O\left(n(\log n)^{1-a}(\log \log n)^{3+a}\right). \quad (2.6)$$

The fixed convolution operand remains charged. For every positive gap $a - \delta$, eventually $(\log \log n)^{3+a} \leq (\log n)^{a-\delta}$, proving (2.3). The compiler and all-length reduction are invoked with their hypotheses, not re-proved here. Integer-multiplication bit-network constraints, stopping losses and tape precision estimates are not premises of this transfer.

3. Implementing finite adapters in the array model

The layout argument extends [O], Lemma 2.5. For fixed m , a binary basis change on each of f columns and a selection of r output slots per column can be implemented in $O(2^k)$ word and movement operations. Regard each column as a digit of fixed radix 2^m . A finite table gives that digit's contributions to source and destination addresses, including the gathered rf -bit active field. Precompute displaced tables in $O(f + 1)$ work. Traverse the digit-prefix tree while maintaining address sums; it has fewer than 2^{k+1} nodes. Each extension and leaf requires a fixed number of word operations. This includes both gathering and scattering without scanning all f columns separately for each entry.

Translations are incorporated into the tables. A fixed diagonal quadratic phase with values in $\{1, i, -1, -i\}$ on each column is handled by retaining its accumulated exponent modulo four during the same traversal. Products of these column phases and fixed scalar multipliers are therefore charged linear work. Fixed permutations of the finite role stock and finite cover vertices similarly require a fixed number of array passes. These constants may be enormous.

Finite binary linear algebra, frame representatives, gate coefficients and cover tables are computed constructively once. All of their dimensions are fixed independently of k . Neither free data movement nor free per-entry address reconstruction is assumed. A witness using other adapters must supply an additional cost argument.

4. Exact moment certificates

For a given physical histogram, write

$$\Psi(1 - a) = \sum_r \frac{rn_r}{Wm} \exp\left(a \log \frac{m}{r}\right).$$

The checker uses rational upper and lower logarithm bounds obtained by range reduction and the atanh series. For $0 \leq x < 3$,

$$\sum_{j=0}^6 \frac{x^j}{j!} \leq e^x \leq 1 + x + \frac{x^2}{2(1 - x/3)}. \quad (4.1)$$

The upper bound follows because successive terms from the quadratic term onward have ratio at most $x/3$. Lower and upper logarithm endpoints are rounded outward to a rational grid before use. Every inequality determining acceptance is rational; decimal displays are explanatory only. The earlier round-six certificate uses a looser geometric exponential bound, which remains valid and unchanged.

A result record contains its exact tensor witness a , Fourier choice δ , positive absorption gap, physical histogram, immutable source pins, verifier, and explicit proof dependencies. A lower moment above one at a larger a rejects that value for the same histogram; it is not an optimality theorem over other networks.

5. Versioning and evidence boundaries

The stable source of Sections 1–5 is `paper/framework.md`. Network-specific arguments live under `results/`; `results/registry.json` selects the current result explicitly. The build assembles the selected record into this paper and generates the improvement log. It never selects the largest upstream number automatically.

An incorporated result requires a written discharge of the network contract or an imported formal Fourier theorem, a pinned complete witness, exact moment and absorption checks, a reproducible finite verifier, and a review record stating its limitations. The status **proposed** does not mean independently reviewed or formally verified. The status **formalized** identifies an incorporated upstream theorem with a reproduced Lean project build, axiom audit and statement comparisons; it does not extend that claim to these reusable written arguments or imply independent human review. Candidates remain visibly separate from incorporated results. Earlier proofs, certificates and publication artifacts remain available even after a newer result is selected.

This work is AI-assisted. Additional checks written in this repository are not independent mathematical review. Vendor checkers are identified as such; random exact or modular trials are not described as universal identities. Lean arithmetic certificates are distinguished from a formal network or Fourier theorem. No novelty or priority is claimed for imported network improvements.

The following selected witness supplies the construction-specific arguments and attribution. Its status and source pins belong to that record; future updates need not rewrite the general transfer theorem.

6. Sussman and Boukhalfa’s five-stage Fourier result

6.1 Sources, theorem and status

This record incorporates the community complex circuit exported by Chafik Boukhalfa in CrocSwap PR256 [CB], in Jacob Sussman’s five-stage construction and formal Fourier framework [JS]. The explicit-program package is pinned to `db3f75cdc5f03f3131d48fb220f6bf1958404ff3`; the corresponding Fourier proof is pinned to `fd19e46b728faa414cbfbf91b01a6a0b7903375d`. The latter instantiates Sussman’s framework with the stronger PR233 circuit. These are upstream constructions and theorems. This project’s contribution in this update is their incorporation, source audit and reproducible checks.

The selected exponents are

$$a = \frac{7547361}{10^{10}} = 0.0007547361, \quad \delta = \frac{7547360}{10^{10}} = 0.0007547360, \quad a - \delta = 10^{-10}.$$

In OpenAI’s exact-complex-arithmetic RAM model, the imported theorem states that one fixed program computes the exact DFT for every positive length in

$$T(n) = O\left(n(\log n)^{1-\delta}\right).$$

It includes scalar preparation, array/index work and the supplied-root interface. The root order is computed by a program and is less than $1024n^3$; the solver receives that specified root. Integer

values and indices have a polynomial bound in n . A separate theorem supplies exact convolution. This changes neither the model nor the absence of a practical FFT-speed claim.

The pinned Lean project was rebuilt from source, using the pinned Mathlib cache. The WHT and all-length DFT/convolution statement comparisons passed, and the Fourier and tensor-program theorems report only the standard Lean axioms. The result is selected as **formalized**, referring to that imported theorem. The receipt, exact commands and limits are recorded in `verification/FIVE_STAGE_AUDIT.md`; independent human review remains pending.

6.2 The explicit complex program

The underlying lineage is icekylinx’s paired cubes and source-assisted flow, eumemic’s PR168 v4 modules and frames, Avi Eisenberg’s PR191/193/194 reconstruction and pairing, and Boukhalfa’s PR200/233 maximum-weight reuse pairing. PR233 retains the operation frames and replaces the reuse pairing. Sussman supplies the general frame formalization, certificate format and checker; Boukhalfa’s PR256 emitter lowers the flow witness and exact local lifts into that format. an664’s sharing principle and the earlier carrier, producer and endpoint work retain their source credits.

The certificate has $h = 22$, $v = 1320$, $R = 9412$ helper slots, 12052 numbered registers, 18248 frames, and 52300 gates across its two phases. Its 69683 transformation blocks have total rank 262944. The retained-center contribution is 440 units of rank. These counts include the retained-center copies; no center-return or dirty-register work is silently discarded.

The Python reference checker verifies nested frame labels, gate shape and scatter conditions, the block ledger, and the exact response of every source column. It checks that each target receives the required source and that the sources are restored. This scalar check alone is not a replay of every dirty input column. The formal invocation theorem derives the dirty-response subtraction and inverse cleanup from the checked invertible program and the scalar identities. The full proof, rather than the Python source-column test alone, establishes the arbitrary-scratch behavior.

The certificates supplied to the Python and Lean pipelines have identical mathematical fields. Only their descriptive provenance string differs. The local verifier checks that equality, as well as the source hashes and the numerical parameters used by the formal rate theorem.

6.3 Five-stage construction and its ledger

Sussman’s bridged construction gives each source pair a twin and arranges compatible labels so that two exchanges use five helper invocations in place of six. The helpers are shared across completed stages. Per finite-cover vertex,

$$m = 5h = 110, \quad W_0 = 4v + R = 14692.$$

If H_{inv} is the complete block histogram of one certified invocation, the five-stage histogram is

$$H_5 = 5H_{\text{inv}} + 2v(e_{42} + e_{21} + e_{46} + e_4),$$

where e_r denotes one child of width r . The extra terms are the twins’ remaining frame transitions. The resulting per-vertex histogram is:

r	n_r	r	n_r	r	n_r
1	145890	9	2020	17	195
2	63335	10	4025	18	13200
3	49940	11	1765	19	7205
4	21835	12	5705	20	110
5	9880	13	840	21	2640
6	7455	14	3625	42	2640
7	3210	15	3025	46	2640
8	7390	16	405		

Thus there are 358975 children, the largest width is $46 < 110$, and

$$\sum_r r n_r = 1613040 = 110 \cdot 14692 - 3080.$$

The cover multiplier cancels in the normalized moment but remains part of the finite algorithm's stock and constants. The imported proof constructs the network over the orthogonal group in dimension 110 and proves its routing and adapter costs. Neither this integration nor the upstream implementation enumerates that enormous cover on actual arrays.

6.4 Exact rate and the formal tensor-to-Fourier connection

Our rational checker verifies

$$\Psi(1 - a) = \frac{1}{14692} \sum_r n_r (r/110)^{1-a} < 1, \quad 1 - \Psi(1 - a) > 8.05 \cdot 10^{-10}.$$

The imported recursion engine uses a finite-fill correction with $s = 40$. We separately check its actual rate premise:

$$(1 - 2^{-40})\Psi(1 - a) + 2^{-40} \left((109/110)^{1-a} + (1/110)^{1-a} \right) < 1.$$

The formal lemma `BlockAccounting.fold_B2Gp233x` proves this rate inequality using its own rational bounds. At $a = 7547365/10^{10}$ the ordinary moment is greater than one, providing a negative control for this fixed histogram. This is not a ceiling on other networks.

The formal chain applies the checked certificate to the generalized frame/recursion engine. `RAM.hillsY_program` supplies OpenAI's tensor-program interface, with exponent $1 - a$, for every required mode, color and input batch. The subsequent files under `Work/Fourier233/` reproduce OpenAI's Fourier reduction with the exponent and declaration names changed. Its overhead is absorbed using the positive gap $a - \delta = 10^{-10}$. The final theorem is `OAI.PowerSaving.transform_mainY : DFTGoalY; convolution_mainY` proves the companion convolution statement.

The challenge file is OpenAI's complete uniform Fourier statement with its exponent and seven declaration names changed, plus explanatory comments. Our source comparison checks those limited differences. The theorem comparison additionally checks the declarations on which the statement depends and the axioms used by its proof. The original model, supplied root, preparation costs and integer bounds remain in the statement.

Sections 1–5 give this paper’s reusable written transfer argument. The selected result is also available directly through the imported formal theorem and its own recursion engine; the local Python moment check is not presented as a replacement proof of that theorem.

6.5 Reproduction and attribution

Run `python3 verification/verify_five_stage.py` for source hashes, exact scalar/label checks, the histogram, both rate inequalities, the absorption gap and binding to the Lean sources. Full PR256 regeneration additionally requires its pinned NumPy/SciPy environment. The complete upstream Lean source is preserved in `verification/vendor/five_stage/fourier-proof-source.tar.gz`, with immutable repository pins and licenses in the accompanying manifest.

The separate `verification/verify_five_stage_lean.py` rebuilds the formal development with Lean 4.34.1 and the pinned Mathlib revision, regenerates the 41 certificate/proof files and 14 Fourier files, checks the reported axioms, and runs the upstream theorem/definition comparator. Its receipt distinguishes fresh project compilation from the downloaded Mathlib cache. These checks do not constitute independent human review, an execution of the DFT on large arrays, or a run of an independent proof kernel.

Sussman receives credit for the five-stage geometry and formal framework; Boukhalfa for the stronger pairing, explicit program and Fourier instantiation; Eisenberg, icekylinx, eumemic and the named predecessors for the underlying circuit and algorithms. SovereignSteak’s PR250 and hcg890’s PR234 are credited in the incoming package’s five-stage composition and ledger lineage. OpenAI supplies the original Fourier model and all-length reduction. The source audit preserves the broader community and our earlier Jain-based history. This update does not claim a newly discovered exponent or authorship of the imported proof.

Acknowledgements and source lineage

We acknowledge OpenAI for the original #109 and #130 research and the exact Fourier framework [O, O109]; Douglas Colkitt for early network and parameter improvements, the independently sized complex network that informed our initial paper, and community review and integration [D]; Rohan Arun for fixed-basis/corner compositions and weighted carrier matching, the latter explicitly acknowledged in Jain’s source lineage [RA]; and Swapnil Jain for the complex witnesses adopted in our rounds six, ten and eleven [J] (see the versioned result records). This introductory order records research context rather than a ranking or a linear chain of authorship. OpenAI supplies the tensor-to-Fourier compiler and all-length reduction.

The adopted constructions also depend on icekylinx’s whole-residual batching, copied centers, generalized Clifford frames, three-stage covers, paired cubes and retirement reuse; eumemic’s batching, producers, modules, frame descent and late compensation; an664’s completed-core sharing; James Chang’s birth reuse and terminal-target compiler; and DaysSky’s carrier extensions. Aurel Prosz’s two-stage topology and endpoint corrections are predecessors of the earlier Jain witness. Chafik Boukhalfa’s terminal paired-cube work is already credited in Jain’s round-eleven source, alongside his related local-circuit configuration. Avi Eisenberg, Zhihao Chen, Andrew Barnes, David Leen and the other authors named in the preserved notices retain their specific predecessor credits.

The selected five-stage result incorporates Jacob Sussman’s general frame formalization, five-stage construction and formal all-length DFT theorem [JS], with Chafik Boukhalfa’s stronger pairing,

explicit certificate and Fourier instantiation [CB]. Its circuit passes through icekylinx’s source-assisted flow, eumemic’s v4 modules and Avi Eisenberg’s reconstruction and pairing; SovereignSteak and hcg890 retain their cited composition and ledger credits. We reproduced the pinned upstream Lean proof and theorem comparisons. Those network and formalization contributions belong to their authors. Related Fourier work includes eumemic’s written batched-network transfer [EF] and danadran01’s Lean development [DF]. Our project contributes its earlier written conditional transfers, this incorporation, the attribution audit and additional local checks; it does not claim discovery of the imported exponent or authorship of OpenAI’s reduction.

The repository’s `ATTRIBUTION.md` records immutable revisions, incorporated, historical and related dependencies, source discrepancies and the broader contributor ledger. Contributor-specific licenses and AI-assistance notices are preserved. Acknowledgement does not imply collaboration, endorsement or independent mathematical review.

Shared source references

[O] OpenAI. *An explicit power saving for the exact discrete Fourier transform*, September 25, 2026. Lemmas 2.2 and 2.5, Proposition 2.4, Theorem 2.6, Proposition 4.2 and Section 5.4. [Public manuscript](#). The imported formal development uses [Lean sources](#) at commit `fd4aeeb2ee4fc729c18d98444fed42fd0529eeeb`; the 89 imported modules, original challenge and toolchain were checked byte for byte against that revision.

[D] Douglas Colkitt. *A sharper exponent for integer multiplication*. [Independently sized complex network](#), commit `6e564879f51ae16f23d392e9e196c605f36d90df`; [community contribution and review record](#). Project maintenance does not imply sole authorship of community results.

[RA] Rohan Arun. [PR44: weighted carrier matching](#), commit `1a21794eb5384db0e96d7bc2b11aa6f28cff7a85`; earlier [PR39 fixed-basis/corner composition](#). Jain’s notices explicitly acknowledge the matching precedent. The later bit-side improvements are not premises of this Fourier transfer.

[J] Swapnil Jain. *Integer multiplication: complex-network witnesses*. [Round-eleven construction and predecessor notices](#), commit `1a580dcc91dad5fbcffb5c80b7641a96bfa24e16`; [round-six source](#), commit `f2176bc1124821bf17eb63725bd366d7bdc020a3`. The round-ten and round-eleven records preserve their separate source pins and contribution boundaries.

[O109] OpenAI. *Integer multiplication below $n \log n$* , September 23, 2026. [Pinned manuscript sources](#).

[EF] eumemic. *Exact DFT bounds*, commit `6f87d1a0c9ae462cdcb2034a1361c87623221e66`. [Written batched-network transfer](#).

[DF] danadran01. *Exact DFT power saving*, revision `1a7b25e`. [Related Lean development](#). Not rebuilt in this attribution audit.

[JS] Jacob Sussman. *Walsh-Hadamard and all-length Fourier power savings*, commit `f010392c923279e3dd59ef3aa5fedad23affc5fd`. [Formal development and related-work record](#).

[CB] Chafik Boukhalfa. [PR256: explicit numbered complex program](#), commit `db3f75cdc5f03f3131d48fb220f6bf1958404ff3`; [stronger Fourier instantiation in Sussman’s framework](#), commit `fd19e46b728faa414cbfbf91b01a6a0b7903375d`. These packages retain the community circuit authors’ notices.

Incorporated result history

These are project records, not priority claims. Earlier proposed transfers retain their original status; formalized denotes an incorporated upstream theorem with a reproduced Lean build and statement checks. Ratios compare exponent savings, not runtimes.

Date	Witness	Status	Fourier saving
2026-10-08	legacy-shared-sum	historical-proposed	0.00000000055
2026-10-08	round6	proposed	0.000073
2026-10-09	round10	proposed	0.00061
2026-10-09	round11	proposed	0.00067
2026-10-09	five-stage-pr233	formalized	0.000754736

The full log, record links and update workflow are in `IMPROVEMENTS.md` and `results/README.md`. Earlier assembled papers remain under `archive/`.